

# 收集器

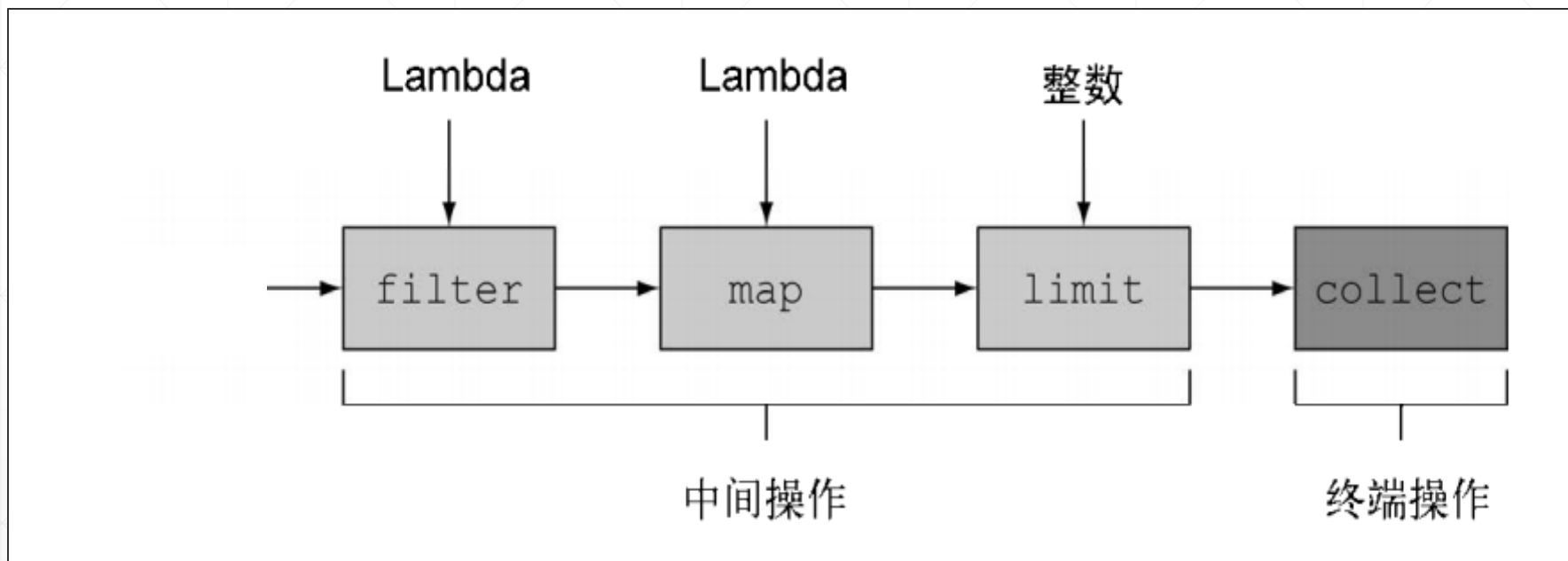
---

北京理工大学计算机学院  
金旭亮



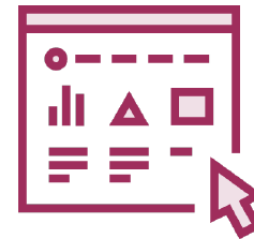
# 什么叫“收集”？

collect顺次收集流中的所有元素，进行某种处理之后得到一个结果，它是一个终端操作。



完成“收集”操作的对象，称为“收集器”。

# 常用的四类收集器



1

将流转换为某种集合，比如`Collectors.toList()`收集器

2

将流元素归约和汇总为一个值，比如`Collectors.counting()`收集器

3

使用`Collectors.groupingBy()`收集器将流元素分组

4

使用`Collectors.partitioningBy()`收集器将元素分区

# 关于收集器



- ✓ Collector 会对元素应用一个转换函数（可以是不体现任何效果的恒等转换，例如 `toList`），并将结果累积在一个数据结构中，从而产生这一过程的最终输出。
- ✓ 但凡要把流中所有的元素合并成一个结果（或转换为另一种形式）时，就可以用收集器。
- ✓ JDK的`java.util.stream`包中有一个`Collectors`类，它里面提供了诸多现成的收集器，可以直接使用。
- ✓ 我们也可以定义自己的收集器，不过需要这么干的场合不多,并且实现起来也比较复杂。

# “集合类型”收集器

最常用的两种集合收集器，是`toList()`和`toSet()`：

```
//使用收集器将Stream转换为Set，从而自动地消除重复元素
static void distinctWithSet() {
    var nums :List<Integer> = List.of(1, 1, 3, 5, 9, 3, 8);
    Set<Integer> distinctNumbers = nums.stream()
        .collect(Collectors.toSet());

    System.out.println(distinctNumbers);
}
```

# 字符串收集器

将集合中的各个数字使用 “,” 分隔，连接为一个字符串：

```
var numbers : List<Integer> = List.of(1, 2, 3, 4, 5,  
    6, 5, 4, 3, 2, 1);  
//字符串连接收集器  
var numString : String = numbers.stream()  
    .map(num -> String.valueOf(num))  
    .collect(Collectors.joining(delimiter: ", "));  
System.out.println(numString);
```

# 练习



`Collectors.joining()`方法有几个重载形式，其中有一个比较有趣：

```
public static Collector<CharSequence, ?, String> joining(
    CharSequence delimiter,
    CharSequence prefix,
    CharSequence suffix)
```

请编写一个示例，看看以上方法是如何生成最终结果的，你还可以如何利用它来格式化最终的结果。

# mapping收集器：先转换，再收集

这一收集器接收两个参数：

```
public static <T, U, A, R> Collector<T, ?, R> mapping(  
    Function<? super T, ? extends U> mapper,  
    Collector<? super U, A, R> downstream)
```

示例：

```
static void useMappingCollector() {  
    var students : List<Student> = Student.getExampleStudents();  
    //用于提取Student对象的名字字段值  
    Function<Student, String> getStudentName = Student::getName;  
    //将每个Student对象先转换为字符串，再收集到List中  
    List<String> nameList = students  
        .stream()  
        .collect(Collectors.mapping(getStudentName, Collectors.toList()));  
    System.out.println(nameList);  
}
```



# minBy收集器

```
static void useMinByCollector() {  
    var students : List<Student> = Student.getExampleStudents();  
    //提取出学生的Gpa成绩  
    var gpaComparator : Comparator<Student> = Comparator.comparing(Student::getGpa);  
    //获取Gpa最低的那名学生的信息  
    var minResult : Optional<Student> = students.stream().collect(  
        Collectors.minBy(  
            gpaComparator  
        ));  
    minResult.ifPresent(System.out::println);  
}
```



类似地，还有一个maxBy收集器。

# 统计相关的收集器

//统计相关的收集器

```
static void useStatisticsCollectors() {  
    var numbers : List<Integer> = List.of(1, 2, 3, 4, 5, 6,  
        5, 4, 3, 2, 1);  
    //计数收集器  
    long count = numbers.stream().collect(Collectors.counting());  
    System.out.println(count);  
    //最大值收集器  
    var max : Optional<Integer> = numbers.stream()  
        .collect(Collectors.maxBy(Integer::compareTo));  
    System.out.println(max.get());  
}
```

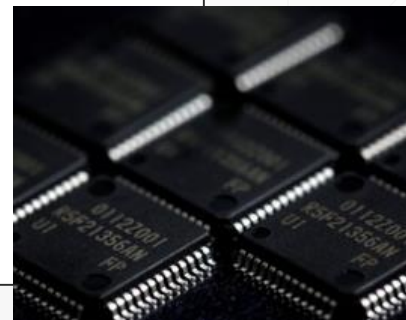
//整数求和收集器

```
int sum = numbers.stream()  
    .collect(Collectors.summingInt(Integer::intValue));  
System.out.println(sum); //求和  
//平均值收集器  
double average = numbers.stream()  
    .collect(Collectors.averagingInt(Integer::intValue));  
System.out.println(average);  
//常用统计量收集器  
var statistics : IntSummaryStatistics = numbers.stream()  
    .collect(Collectors.summarizingInt(Integer::intValue));  
System.out.println(statistics);  
}
```

# 简单分组

```
var numbers : List<Integer> = List.of(1, 2, 3, 4, 5,
    6, 5, 4, 3, 2, 1);

//使用groupBy()收集器对数据进行分组，此方法接收的Lambda表达式，
//必须依据输入值输出有限个数的固定值，它将成为分组的标准
var groupResult : Map<String, List<Integer>> = numbers.stream()
    .collect(Collectors.groupingBy(num -> {
        if (num % 2 == 0) {
            return "even"; //分到偶数组
        }
        return "odd"; //分到奇数组
    }));
System.out.println(groupResult);
```



# 先分组,再过滤

//只对值大于50以上的元素进行分组

```
var evenAndOddNumbersGreateThan50 : Map<String, List<Integer>> =  
    IntStream.rangeClosed(1, 100)  
        .boxed()  
        .collect(Collectors.groupingBy(num -> {  
            if (num % 2 == 0) {  
                return "even";  
            }  
            return "odd";  
        }, Collectors.filtering(  
            num -> num.intValue() >= 50,  
            Collectors.toList())));  
  
System.out.println(evenAndOddNumbersGreateThan50);
```

# 分组，同时统计

//分组，同时统计每组元素个数

```
var groupCounting : Map<String, Long> = numbers.stream()
    .collect(Collectors.groupingBy(num -> {
        if (num % 2 == 0) {
            return "even";
        }
        return "odd";
    }, Collectors.counting()));
System.out.println(groupCounting);
```

# 分区

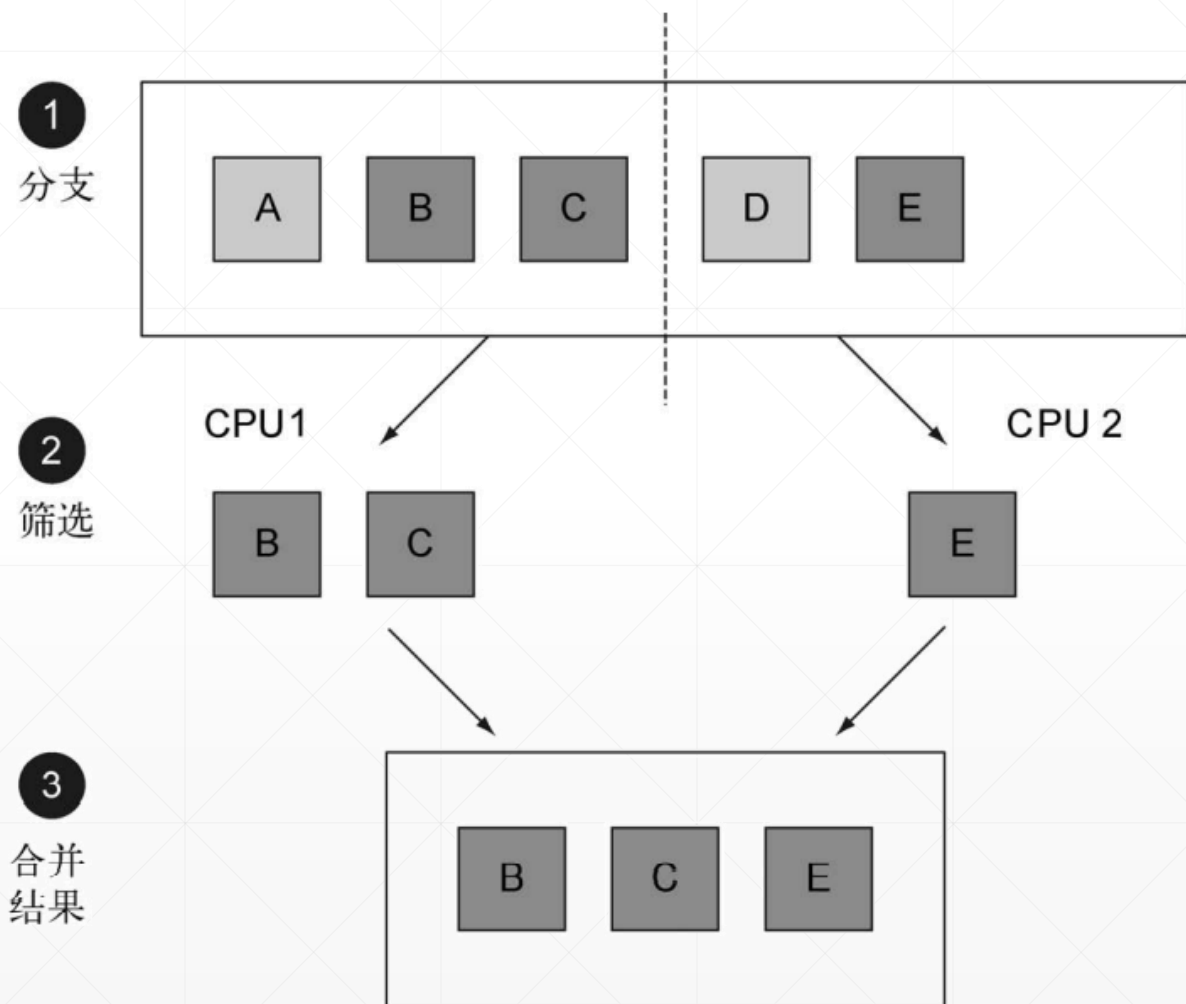


分区是分组的特殊情况：由一个返回一个布尔值的函数作为分类函数，它称分区函数。



分区函数返回一个布尔值，这意味着得到的分组Map的键类型是Boolean（布尔类型），于是它最多可以分为两组——true是一组，false是一组。

# 通过“数据分区”实现并行处理



# 简单分区

```
//依据考试成绩，分为“及格”与“不及格”两组
var doYouPassed : Map<Boolean, List<Integer>> =
    IntStream.rangeClosed(1, 100)
        .boxed()
        .collect(Collectors.partitioningBy(
            score -> score >= 60));
System.out.println(doYouPassed);
```



注意一下，分区是针对引用类型的数据的，所以要先  
把int转换为Integer再处理。

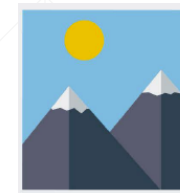


# 先分区,再进一步分组

```
var doYouPassed2 : Map<Boolean, Map<String, List<Integer>>> =  
    IntStream.rangeClosed(1, 100).boxed()  
        .collect(Collectors.partitioningBy(  
            score -> score >= 60,  
            Collectors.groupingBy(score -> {  
                int temp = score / 10;  
                String result = switch (temp) {  
                    case 0, 1, 2, 3, 4, 5 -> "不及格";  
                    case 6 -> "及格";  
                    case 7 -> "中";  
                    case 8 -> "良";  
                    case 9, 10 -> "优";  
                    default -> "无效成绩";  
                });  
                return result;  
            })))  
System.out.println(doYouPassed2);
```

先将学生分成“及格”与“不及格”两个区，然后，再对每个区进行进一步的分组。

# 区分Reduce与Collect



```
List<Integer> numbers = ... ;
```



```
numbers.stream().reduce(...)
```

Reduce的结果类型，是由Stream的类型决定的。



```
numbers.stream().collect(...)
```

Collect，可以返回任意类型的结果